

Program Sarjana, Departemen Teknik Komputer, FTEIC-ITS

# Proyek "2EZ4U APP"

Studi Kasus Proyek Akhir
Struktur Data & Analisa Algoritma (EC234303)
Dosen: Ir. Arta Kusuma Hernanda, S.T., M.T.

Project yang akan dikembangkan adalah **engine backend layanan antar makanan/barang** seperti GoFood/GrabFood/ShopeeFood versi mini.

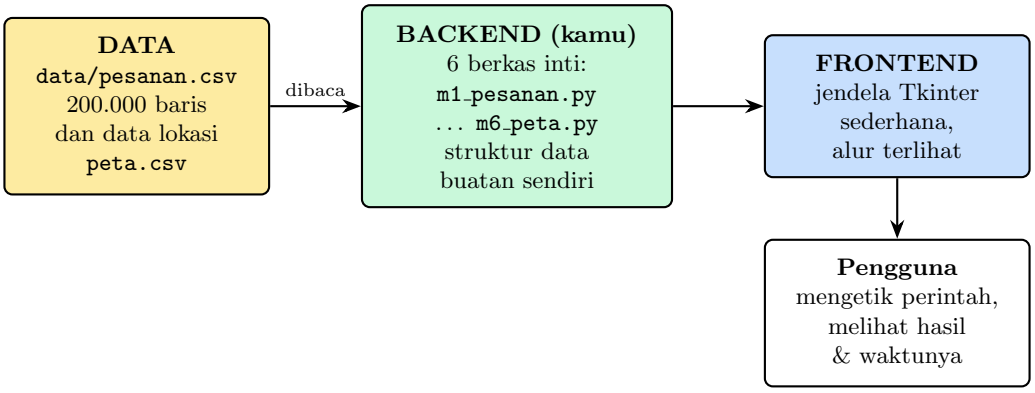
| | | | |
|---|---|---|---|
| **Bentuk** | individu | **Bahasa** | Python 3.11+ |
| **Bobot** | 35% nilai akhir | **Submission** | repo GitHub |
| **Mulai** | minggu 1 | **Demo** | Progres(M9) Final(M17) |

## 1 Apa yang Kamu Develop?

Bayangkan kamu akan mengembangkan aplikasi delivery service dengan trafik user yang cukup tinggi. Aplikasi yang kamu kembangkan terdiri atas tiga layer:

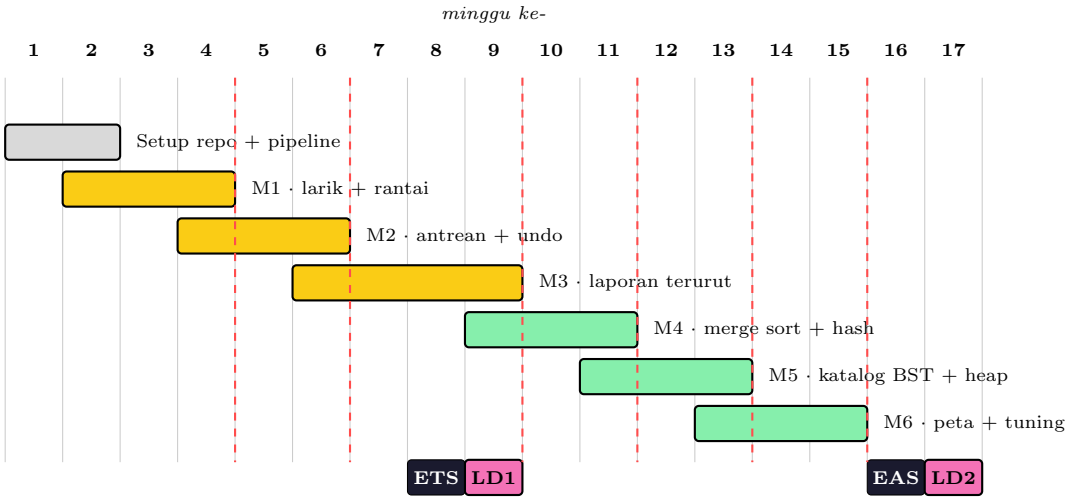
1. **Data** (`data/`): 200.000 data CSV yang diasumsikan sebagai data dalam sistem database aplikasi kalian.
2. **Backend** (`backend/`): **enam fitur utama**, satu fitur untuk tiap milestone, berisi struktur data dan algoritma buatanmu.
3. **Frontend** (`frontend/`): satu jendela **Tkinter** sederhana yang memanggil backend dan **memperlihatkan alurnya** kepada pengguna.

Tampilannya boleh sangat sederhana. Yang dinilai paling besar bukan terkait estiteka UI, melainkan proses implementasi **struktur data dan algoritma**.



# 2 Peta Perjalanan Satu Semester

## 2.1 Timeline (Gantt)




*LD1 = sesi Live Demo Progress (mgg 9) · LD2 = sesi Live Demo Final (mgg 17)*

## 2.2 Ringkasan milestone

| # | **Fitur yang terlihat di antarmuka** | **Berkas backend** |
|---|---|---|
| M1 | array vs linked list: lihat, tambah, hapus pesanan | `m1_pesanan.py` |
| M2 | impelementasi queue dan stack (untuk undo) | `m2_antrean.py` |
| M3 | penerapan berbagai macam jenis sort | `m3_laporan.py` |
| M4 | penerapan berbagai macam jenis search | `m4_pencarian.py` |
| M5 | menu per rentang harga; dispatch prioritas | `m5_katalog.py` |
| M6 | map dan navigasi rute | `m6_peta.py` |

# 3 Aturan Main

1. **Project Individu.**
2. **Dilarang menggunakan instant function yang tersedia.** Seperti: `dict`, `set`, `sorted`, `.sort()`, `heapq`, `bisect`, `collections.*`, literal {}, dan comprehension dict/set. **Boleh**: `list` sebagai array primitif, `tuple`, aritmetika, I/O.
3. **Data disediakan dalam format CSV dengan jumlah 200.000 (data.csv) dan data peta.csv.**
4. **UI python desktop seperti Tkinter lib atau sejenisnya.**
5. **Upload project folder ke repository Github**, Wajib menyertakan README.md yang berisi deskripsi project, cara menjalankan, diagram, screenshot, video demo dan catatan penting lainnya. Link repo Github akan dikumpulkan melalui CEITS/ENIGMA.

# 4 Struktur Repositori

Gambar struktur repo github.

```
2ez4u/
|-- README.md             <- LAPORANMU
|
|-- data/
|   |-- peta.csv
|   |-- pesanan.csv
|
|
|-- backend/
|   |-- m1_pesanan.py        M1: Larik (array) + Rantai (linked list)
|   |-- m2_antrean.py        M2: AntreanMelingkar + Tumpukan (undo)
|   |-- m3_laporan.py        M3: insertion sort + binary search
|   |-- m4_pencarian.py      M4: PetaHash + merge sort
|   |-- m5_katalog.py        M5: Katalog (BST) + MinHeap (prioritas)
|   `-- m6_peta.py           M6: Peta (graf) + BFS + Dijkstra
|
|-- frontend/             <- ANTARMUKA
|   |-- ui.py
|
|
|-- app.py                "python app.py" main application
```

## 4.1 Contoh OOP dalam backend

```python
# ---------- backend/m1_pesanan.py ----------

class Array:
    def append(self, v): ...
    def insert(self, i, v): ...
    def get(self, i): ...
        ......

class LinkList:
    def append(self, v): ...
    def get(self, i): ...
        .....

# ---------- backend/m2_antrean.py ----------
class queue:
    def enqueue(self, x): ...
    def dequeue(self): ...
        ...

class stack:
    def push(self, x): ...
    def pop(self): ...
        ...

# ---------- backend/m3_sort.py ----------
def binary_search(larik, target): ...
....

# ---------- backend/m4_pencarian.py ----------
class PetaHash:
    def hash_teks(self, s): ...
    def put(self, k, v): ...
    def get(self, k): ...
        ....

def merge_sort(larik, kunci): ...

# ---------- backend/m5_katalog.py ----------
class Katalog:
    def tambah(self, nama, harga): ...
    def rentang(self, lo, hi): ...
    def stats(self): ...

class MinHeap:
    def push(self, kunci, nilai): ...
    def pop_min(self): ...
    def stats(self): ...

# ---------- backend/m6_peta.py ----------
class Peta:
    def zona(self, a, hops): ...
    def rute(self, a, b): ...
        ...
```

# 5 Data

Sample data di CSV seperti ini:

| oid | pelanggan | resto | menu | harga | prioritas | t_masuk_detik | t_selesai_detik | status |
|---|---|---|---|---|---|---|---|---|
| O-000001 | Rahmi Simanjuntak | kafe-teknik | roti-bakar | 12000 | 3 | 47985 | | CANCEL |
| O-000002 | Cici Handayani | kedai-kopi | bubur-ayam | 11000 | 3 | 59780 | 62115 | DONE |
| O-000003 | Malika Dabukke | warung-bu-ida | mie-goreng | 13000 | 3 | 39641 | 40026 | DONE |
| O-000004 | Oskar Puspita, S.Sos | kafe-teknik | nasi-padang | 25000 | 2 | 29632 | | CANCEL |
| O-000005 | R. Ratna Susanti, S.Pt | bakso-pak-yon | nasi-campur | 20000 | 1 | 74921 | 77006 | DONE |
| O-000006 | Dr. Raina Firmansyah, S.I.Kom | kafe-teknik | ayam-geprek | 19000 | 3 | 30415 | 31330 | DONE |
| O-000007 | Usyi Dongoran | warung-bu-ida | kentang-goreng | 14000 | 2 | 66262 | 68612 | DONE |
| O-000008 | Tgk. Radika Tampubolon | kedai-kopi | iga-penyet | 27000 | 2 | 31196 | 32651 | DONE |
| O-000009 | Hj. Ani Hartati, S.Pt | kantin-asrama | bubur-ayam | 11000 | 3 | 67723 | 68563 | DONE |
| O-000010 | T. Rafid Usamah | kantin-pusat | sate-ayam | 24000 | 3 | 40015 | 40765 | DONE |

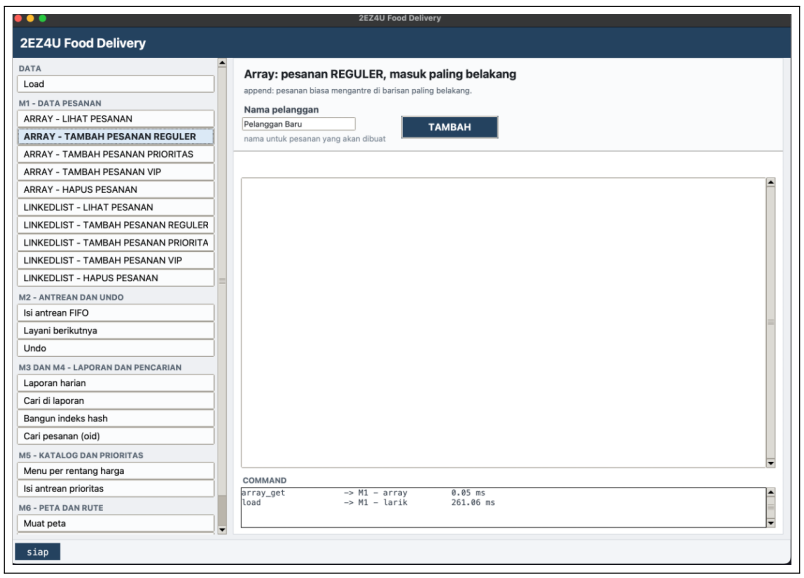
| Kolom | Isi |
|---|---|
| `oid` | ID unik, `O-000001` sampai `O-200000` |
| `pelanggan`, `resto` | data pelanggan dan restoran |
| `menu`, `harga` | data menu dan harga |
| `prioritas` | 1 (VIP), 2 (Prioritas), 3 (Reguler) |
| `t_masuk_detik` | waktu pesanan masuk |
| `t_selesai_detik` | `t_masuk` + lama antar; kosong bila belum selesai |
| `status` | `DONE` / `CANCEL` / `ANTRE` |

# 6 Antarmuka: Satu Jendela Tkinter

Antarmukanya menggunakan **Tkinter**

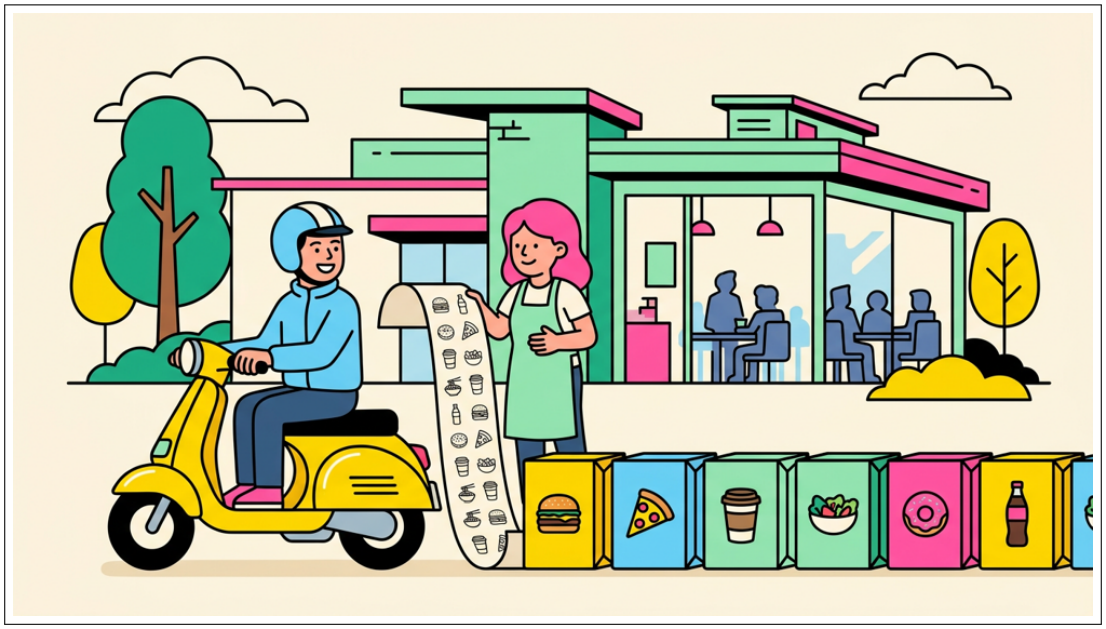
### Contoh layout

Terdapat Tiga bagian: daftar menu di kiri, form parameter dan hasil di kanan, command di bawah. Gambar UI dibawah bisa menajdi refrensi.




Contoh Menu `ARRAY - TAMBAH PESANAN REGULER` dipilih. Form di kanan hanya menampilkan parameter pada menu tersebut, panel `COMMAND` di bawah mencatat perintah yang sudah dijalankan beserta modul dan waktunya.

## 6.1 M1: Data Pesanan, Array vs Linked List kerja mgg 2–4 · deadline akhir mgg 4 · 10%



### Ceritanya

Hari pertama layanan dibuka. Belum ada kurir, belum ada laporan, belum ada peta. Yang paling mendasar dulu: pesanan yang masuk harus **tersimpan dan tidak boleh hilang**, bisa **dibuka kembali** bila ada yang bertanya "pesanan nomor sekian isinya apa?", bisa **ditambah**, dan bisa **dihapus**.

Yang membuat M1 tidak sesederhana kelihatannya adalah aturan bisnisnya: pesanan datang dalam **tiga jenis**. Pesanan **REGULER** (3) mengantre di barisan paling belakang, pesanan **PRIORITAS** (2) menyerobot ke tengah, dan pesanan **VIP** (1) langsung ke urutan pertama. Tiga letak penyisipan yang berbeda inilah yang akan memperlihatkan kekuatan dan kelemahan dua struktur data paling dasar.

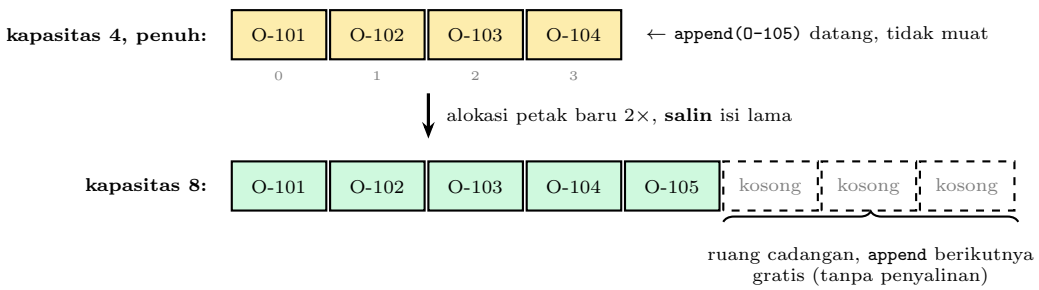
### Backend M1

Data yang sama disimpan pada **dua struktur sekaligus**: sebuah array dan sebuah linked list. Keduanya harus melayani lima operasi yang persis sama, supaya performa bisa diadu langsung dari antarmuka.

| Operasi | Artinya di aplikasi | Array | Linked List |
|---|---|---|---|
| Lihat pesanan ke-$i$ | buka detail satu pesanan | $O(1)$ | $O(n)$ |
| Tambah REGULER | masuk barisan paling belakang | $O(1)$ amortized | $O(1)$ |
| Tambah PRIORITAS | menyerobot ke tengah barisan | $O(n)$ | $O(n)$ telusur |
| Tambah VIP | langsung ke urutan pertama | $O(n)$ | $O(1)$ |
| Hapus pesanan ke-$i$ | pesanan dicabut dari barisan | $O(n)$ | $O(n)$ telusur |

### Bagaimana Array Bekerja (1)

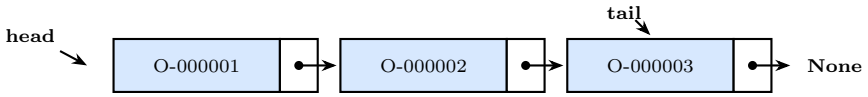
Sebuah array adalah sepetak memori berukuran **tetap**. Ketika penuh, ia tidak bisa "dilebarkan" begitu saja, yang dilakukan adalah menyiapkan petak baru berukuran dua kali lipat, menyalin seluruh isinya, lalu membuang petak lama.



Penyalinan itu mahal ($O(n)$), tetapi **jarang** terjadi: sekali setiap kapasitas berlipat. Bila biayanya dibagi rata ke seluruh `append`, hasilnya konstan. Inilah yang disebut **amortized** $O(1)$. Kalau kamu keliru menyalin *setiap* kali `append`, biayanya menjadi $O(n)$ per operasi dan $O(n^2)$ untuk seluruh pemuatan: benar secara hasil, tetapi **tumbang di uji skala**.

### Bagaimana struktur Linked List Bekerja (2)

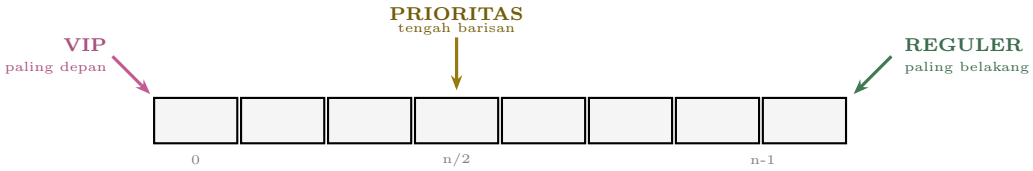
Linked List tidak memakai sepetak memori yang bersambung. Tiap pesanan dibungkus sebuah *simpul* yang menyimpan nilainya dan alamat simpul berikutnya. Karena itu tidak ada yang perlu digeser saat menyisipkan, tetapi juga tidak ada alamat yang bisa dihitung untuk menuju posisi ke-$i$.




*penunjuk `tail` disimpan supaya menambah di belakang tetap $O(1)$*

### Tiga jenis pesanan, tiga letak penyisipan

Di sinilah dua struktur tadi diadu. Aplikasi menerima tiga jenis pesanan, dan yang membedakan biayanya bukan isi pesanannya, melainkan **di mana ia disisipkan**.




Angka berikut diambil dari implementasi referensi pada data 200.000 baris. Perhatikan kolom PRIORITAS: keduanya hampir seimbang, tetapi **karena sebab yang berbeda**.

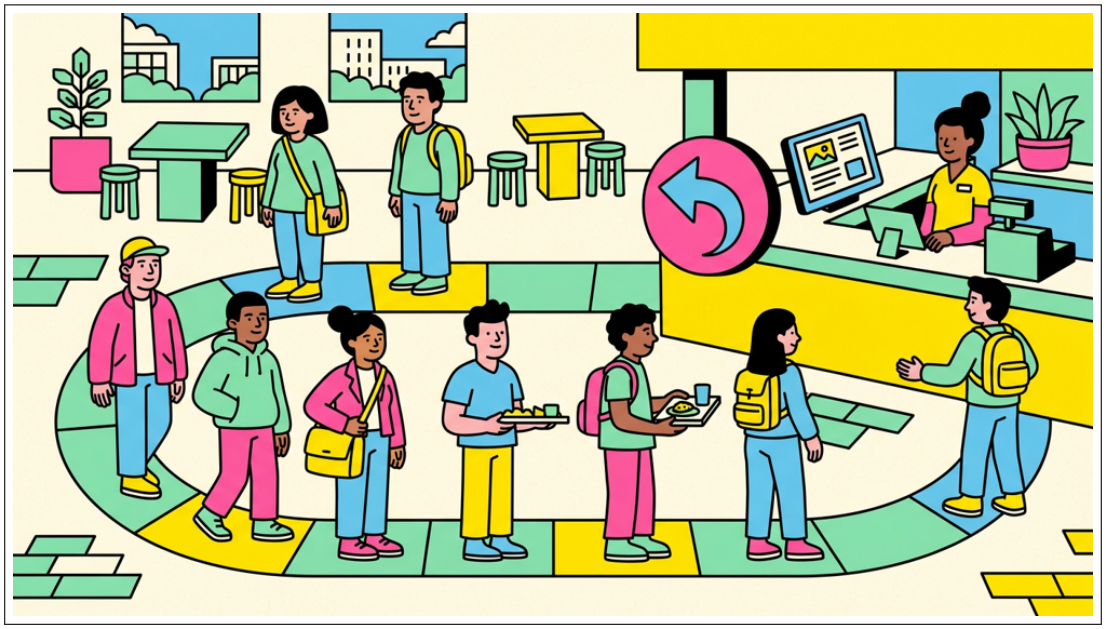
| Jenis | Array | Linked List | Apa yang sebenarnya terjadi |
|---|---|---|---|
| REGULER | 0,02 ms | 0,01 ms | keduanya cukup menempel di ujung belakang |
| PRIORITAS | 3,71 ms | 2,19 ms | array menggeser 100.000 elemen; linked list menelusuri 100.000 simpul lalu menyambung sekali |
| VIP | 6,82 ms | 0,00 ms | array menggeser seluruh isi; linked list hanya mengubah satu penunjuk |

### Fitur M1

| Fitur Array Method | Linked List Method | Keterangan |
|---|---|---|
| ARRAY - LIHAT PESANAN | LINKED LIST - LIHAT PESANAN | Melihat pesanan yang tersedia |
| ARRAY - TAMBAH PESANAN REGULER | LINKED LIST - TAMBAH PESANAN REGULER | Menambahkan pesanan biasa |
| ARRAY - TAMBAH PESANAN PRIORITAS | LINKED LIST - TAMBAH PESANAN PRIORITAS | Menambahkan pesanan prioritas |
| ARRAY - TAMBAH PESANAN VIP | LINKED LIST - TAMBAH PESANAN VIP | Menambahkan pesanan VIP |
| ARRAY - HAPUS PESANAN | LINKED LIST - HAPUS PESANAN | Menghapus pesanan dari antrean |

## 6.2 M2: Stack & Queue

kerja mgg 4–6 · deadline akhir mgg 6 · 15%



### Ceritanya

Dua persoalan muncul sekaligus di warung.

Pertama, di **kasir**. Struk ditulis apa adanya seperti orang menghitung: `( 3 * 12000 ) + ( 2 * 8500 ) - 5000`. Bentuk itu disebut **infix**, dan komputer tidak bisa menghitungnya dari kiri ke kanan begitu saja karena ada prioritas operator dan tanda kurung. Jalan keluarnya: ubah dulu ke **postfix**, baru dihitung. Kedua tahap itu memakai alat yang sama, yaitu **stack**.

Kedua, di **dapur**. Kurir bingung, yang mana dulu? Aturannya sederhana: siapa datang duluan, dilayani duluan (**FIFO**). Dan karena kasir manusia sesekali salah pencet, dibutuhkan tombol **undo** yang mengembalikan keadaan seperti sebelum aksi terakhir.

Jadi M2 berisi dua struktur yang berlawanan sifatnya: **stack** (LIFO, yang terakhir masuk keluar duluan) untuk kasir dan undo, serta **queue** (FIFO) untuk urutan layanan.

### Lima hal yang harus bisa dilakukan backend setelah M2

- `ke_postfix(ekspresi)` mengubah struk infix menjadi postfix memakai stack operator.
- `hitung(postfix)` mengevaluasi postfix memakai stack angka, dan mengembalikan totalnya.
- `undo_last()` membatalkan efek aksi terakhir, dan tetap konsisten dengan `history()`.
- `redo_last()` mengerjakan ulang aksi yang baru saja dibatalkan. Aksi **baru** apa pun mengosongkan tumpukan redo.
- `enqueue(x)` menambah pesanan di belakang antrean, $O(1)$.
- `dequeue()` ada **dua versi**: versi naif di atas larik biasa, dan versi melingkar. Keduanya wajib ada dan wajib bisa dibandingkan, karena justru selisihnyalah yang dinilai.

**Kenapa versi naif tidak boleh dibuang?** Karena tanpa pembanding, kalimat "antrean melingkar itu $O(1)$" hanya jadi hafalan. Dengan dua tombol yang mengerjakan hal yang sama pada data yang sama, selisihnya terbaca sendiri di layar. Isi kedua antrean dengan pesanan yang sama persis, supaya perbandingannya jujur.

### Bagian A (1): kasir, dari infix ke postfix lalu ke total

Aturan konversinya satu paragraf: angka langsung ke keluaran; tanda `(` didorong ke stack; tanda `)` mengeluarkan isi stack sampai bertemu `(` lalu membuang `(` itu; operator mengeluarkan dulu operator di puncak yang **lebih kuat atau sama kuat** sebelum ia sendiri didorong; setelah token habis, seluruh sisa stack dikeluarkan. Prioritas: `*` dan `/` lebih kuat daripada `+` dan `-`.

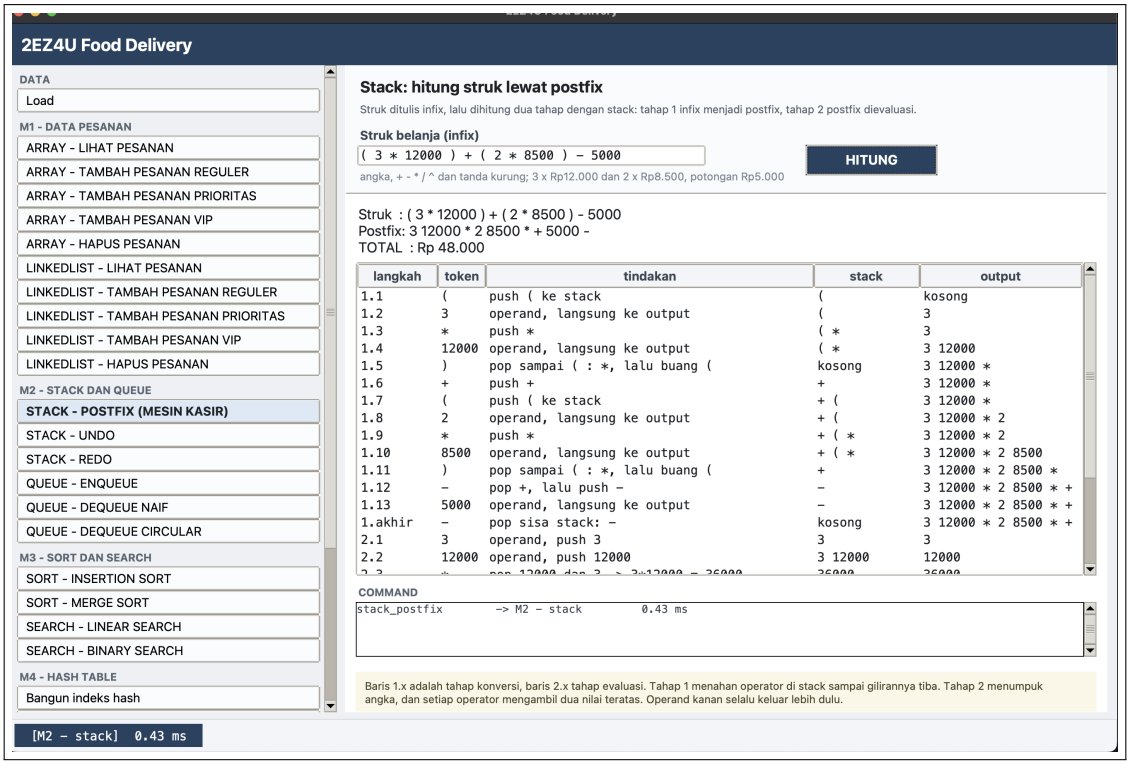
```
Tahap 1  ( 3 * 12000 ) + ( 2 * 8500 ) - 5000   ->  postfix

  token    tindakan                      stack     keluaran
  -------  ----------------------------  --------  --------------------------
  (        push (                        (
  3        angka, langsung keluar        (         3
  *        push *                        ( *       3
  12000    angka, langsung keluar        ( *       3 12000
  )        pop sampai ( , buang (        kosong    3 12000 *
  +        push +                        +         3 12000 *
  (        push (                        + (       3 12000 *
  2        angka, langsung keluar        + (       3 12000 * 2
  *        push *                        + ( *     3 12000 * 2
  8500     angka, langsung keluar        + ( *     3 12000 * 2 8500
  )        pop sampai ( , buang (        +         3 12000 * 2 8500 *
  -        pop + (sama kuat), push -     -         3 12000 * 2 8500 * +
  5000     angka, langsung keluar        -         3 12000 * 2 8500 * + 5000
  akhir    pop sisa stack                kosong    3 12000 * 2 8500 * + 5000 -

Tahap 2  evaluasi postfix                        stack nilai
  -------  ----------------------------------    ----------------------------
  3 12000  push, push                             3 12000
  *        pop 12000 dan 3  -> 3*12000 = 36000    36000
  2 8500   push, push                             36000 2 8500
  *        pop 8500 dan 2   -> 2*8500 = 17000     36000 17000
  +        pop dua teratas  -> 36000+17000        53000
  5000     push                                   53000 5000
  -        pop 5000 dan 53000 -> 53000-5000       48000    <- TOTAL Rp 48.000
```

**Urutan pop menentukan benar-salahnya.** Pada tahap 2, yang keluar *pertama* adalah operand **kanan**. Jadi untuk `-`: `b = pop()` dulu, baru `a = pop()`, dan yang dihitung `a - b`. Kalau dibalik, `+` dan `*` tetap benar karena komutatif, tetapi `-` dan `/` langsung salah dan bugnya susah terlihat. Uji khusus dengan ekspresi yang memuat pengurangan dan pembagian.

Begini rupanya pada implementasi acuan: kedua tahap tadi ditampilkan sebagai satu tabel langkah, jadi isi stack bisa diikuti baris demi baris.

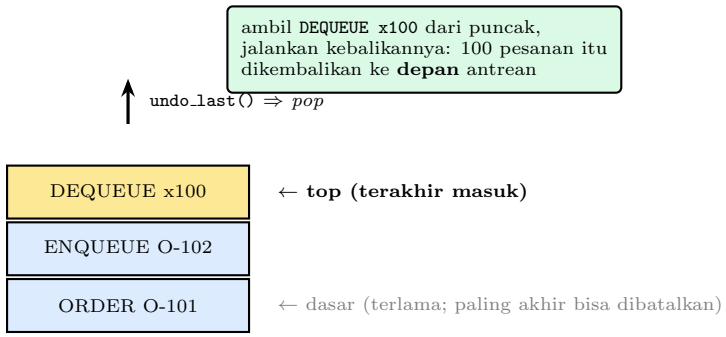



Menu `STACK - POSTFIX (MESIN KASIR)` untuk struk `( 3 * 12000 ) + ( 2 * 8500 ) - 5000`. Baris `1.x` adalah tahap konversi dan baris `2.x` tahap evaluasi; perhatikan langkah 1.5 dan 1.11, tanda `(` hanya mengatur stack dan tidak pernah muncul di kolom `output`, dan hasil akhirnya TOTAL Rp48.000, persis seperti transkrip di atas.

### Bagian A (2): stack undo dan redo

Setiap aksi yang mengubah keadaan didorong (*push*) ke tumpukan **undo** bersama informasi secukupnya untuk membatalkannya. `undo_last()` mengambil elemen teratas (*pop*) lalu menjalankan kebalikannya.

Tumpukan kedua, **redo**, menampung aksi yang baru saja dibatalkan. `redo_last()` mengambil elemen teratas tumpukan itu lalu mengerjakan *ulang aksi aslinya*, bukan kebalikannya, karena keadaan antrean saat itu sudah dikembalikan oleh undo.



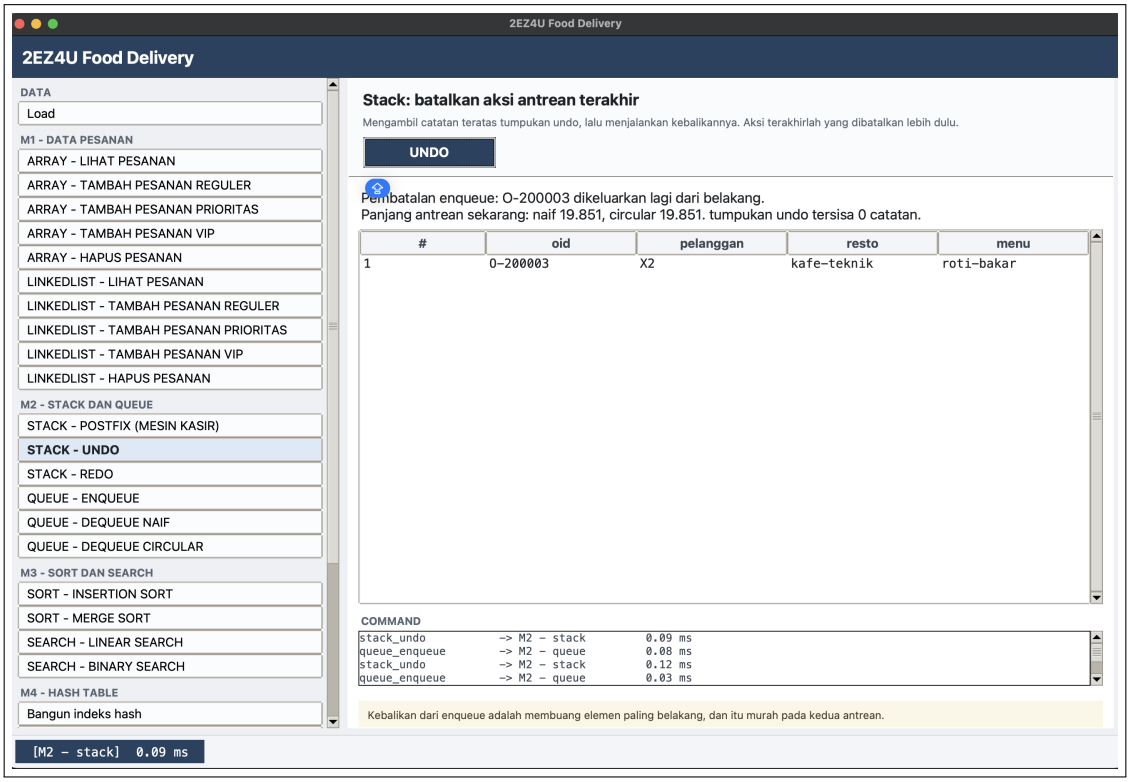

| Aksi yang tercatat | Kebalikannya saat undo |
|---|---|
| `ORDER oid` | hapus/tandai batal pesanan `oid` yang baru ditambahkan |
| `ENQUEUE oid` | keluarkan lagi `oid` dari *belakang* antrean |
| `DEQUEUE daftar` | kembalikan seluruh isinya ke *depan* antrean (bukan ke belakang!) |

| Kejadian | Yang dilakukan pada kedua tumpukan |
|---|---|
| aksi baru (enqueue/dequeue) | `push` ke **undo**; **kosongkan redo** |
| `undo_last()` | `pop` dari undo; jalankan kebalikannya; `push` ke **redo** |
| `redo_last()` | `pop` dari redo; kerjakan lagi aksi aslinya; `push` kembali ke **undo** |

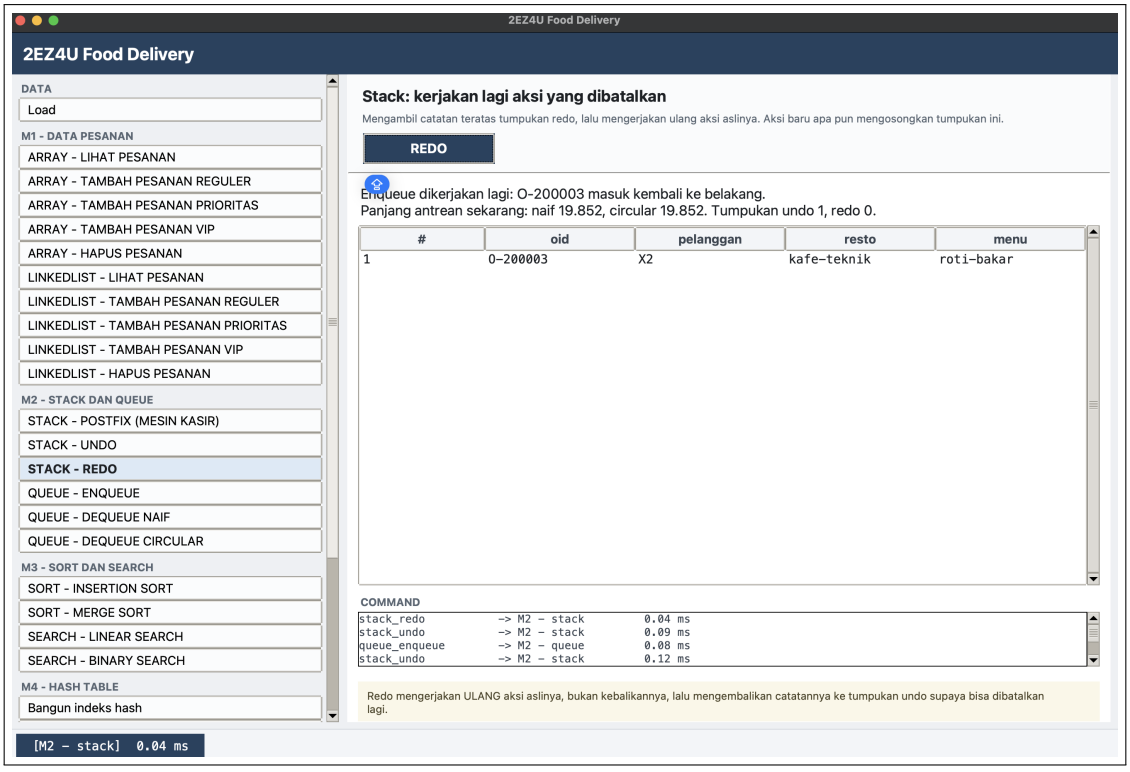
**Kenapa redo harus dikosongkan?** Setelah beberapa kali undo, pengguna berada di titik lama dalam riwayat. Bila dari situ ia mengerjakan aksi *baru*, riwayatnya bercabang dan aksi yang tadi dibatalkan sudah tidak masuk akal untuk dikerjakan ulang. Kalau tumpukan redo tidak dikosongkan, tombol redo akan menempelkan potongan riwayat lama ke keadaan baru, dan datanya kacau **tanpa pesan galat apa pun**.

Catatan undo menyimpan **daftar** pesanan, bukan hitungannya saja. Kalau yang disimpan hanya angka "100", kamu tidak punya bahan untuk mengembalikannya. Aturan umum: simpan yang dibutuhkan untuk membalikkan, bukan yang dibutuhkan untuk menjelaskan.

Dua tangkapan layar berikut adalah pasangan undo dan redo atas satu aksi `ENQUEUE` yang sama, jadi angkanya bisa dibandingkan langsung.



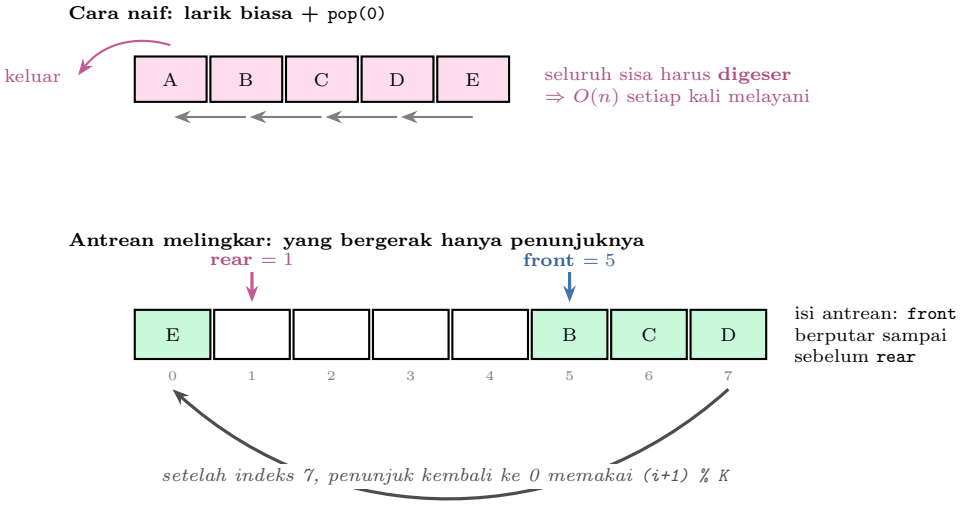

Menu `STACK - UNDO` sesudah satu kali enqueue. Kebalikan dari enqueue dijalankan: `O-200003` dikeluarkan lagi dari *belakang*, panjang kedua antrean turun dari 19.852 menjadi 19.851, dan tumpukan undo tersisa 0 catatan sehingga tombolnya tidak punya apa-apa lagi untuk dibatalkan.




Menu `STACK - REDO` tepat sesudah undo di atas. Yang dikerjakan adalah *aksi aslinya*, bukan kebalikannya: `O-200003` masuk kembali ke belakang, panjang antrean naik lagi ke 19.852, dan hitungan di baris kedua berbalik menjadi tumpukan undo 1, redo 0.

## Bagian B: dua versi dequeue, dan kenapa yang satu harus melingkar

Cara paling naif membuat antrean adalah memakai larik biasa lalu mengambil elemen depan dengan `pop(0)`. Masalahnya: begitu elemen depan diambil, **seluruh sisa elemen harus digeser satu langkah ke kiri**. Untuk dua puluh ribu pesanan, itu dua puluh ribu penggeseran untuk satu kali layanan.




Melayani satu pesanan = `front` maju satu langkah. Tidak ada elemen yang digeser ⇒ $O(1)$.

Pada antrean melingkar, isi antrean adalah potongan dari `front` berputar sampai sebelum `rear`. Ketika penunjuk mencapai ujung larik, ia kembali ke indeks 0. Itulah gunanya operasi modulo.

| Operasi | Yang dikerjakan | Penjaga |
|---|---|---|
| `enqueue(x)` | `data[rear] = x; rear = (rear+1) % K; count += 1` | tolak/tumbuh bila `count == K` |
| `dequeue()` | `x = data[front]; front = (front+1) % K; count -= 1` | kembalikan `None` bila `count == 0` |
| `is_empty()` | `count == 0` | **jangan** pakai `front == rear` saja |

**Kenapa harus ada `count`?** Kalau kamu hanya menyimpan `front` dan `rear`, keadaan *kosong* dan keadaan *penuh* sama-sama membuat `front == rear`, tidak bisa dibedakan. Dua obat yang lazim: simpan pencacah `count` (paling sederhana, disarankan), atau sengaja menyisakan satu slot kosong. Pilih satu, lalu **tulis alasannya di README**.

### Contoh nyata

```
> stack_postfix   ( 3 * 12000 ) + ( 2 * 8500 ) - 5000
  postfix: 3 12000 * 2 8500 * + 5000 -
  TOTAL  : Rp 48.000                                  [M2 - stack]  0,09 ms

> queue_enqueue   Pelanggan Baru
  19.850 pesanan berstatus ANTRE masuk ke KEDUA antrean
  O-200001 masuk paling belakang                      [M2 - queue]  0,01 ms

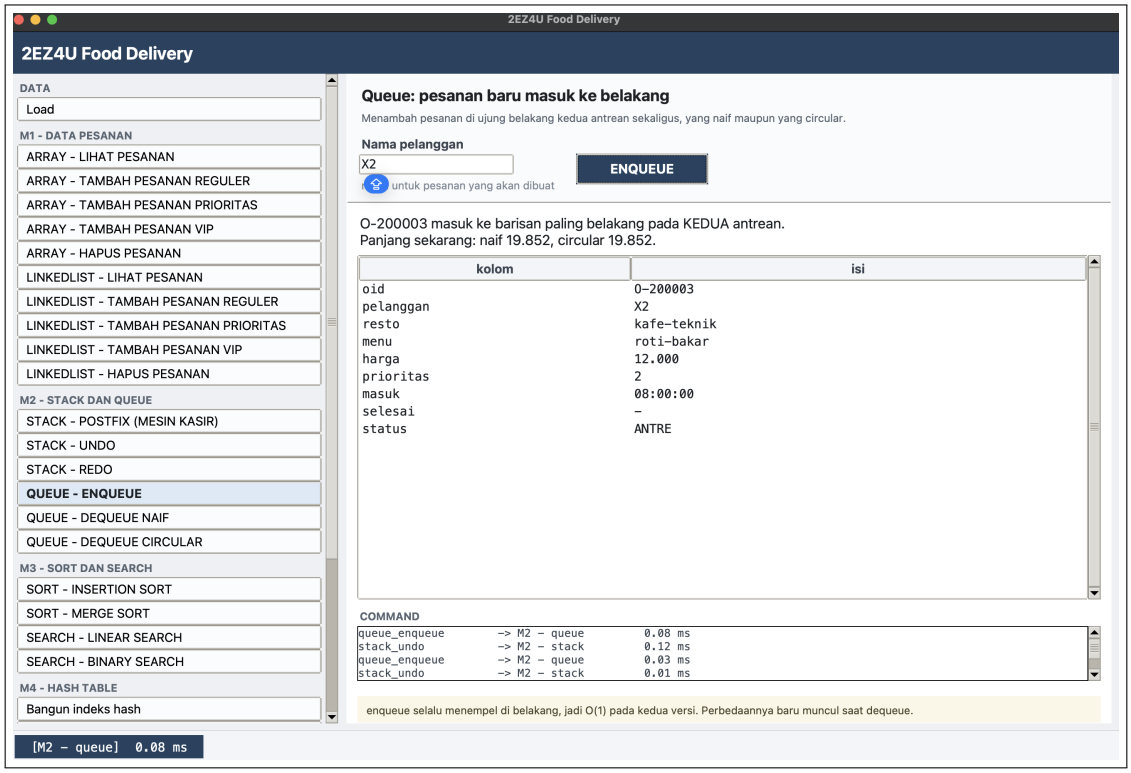
> queue_dequeue_naive      1
  1 pesanan keluar | 19.850 elemen digeser       [M2 - queue naif]      1,07 ms
> queue_dequeue_circular   1
  1 pesanan keluar | 0 elemen digeser            [M2 - queue circular]  0,01 ms

> queue_dequeue_naive      100
  100 pesanan keluar | 1.979.950 elemen digeser  [M2 - queue naif]     93,27 ms
> queue_dequeue_circular   100
  100 pesanan keluar | 0 elemen digeser          [M2 - queue circular]  0,03 ms

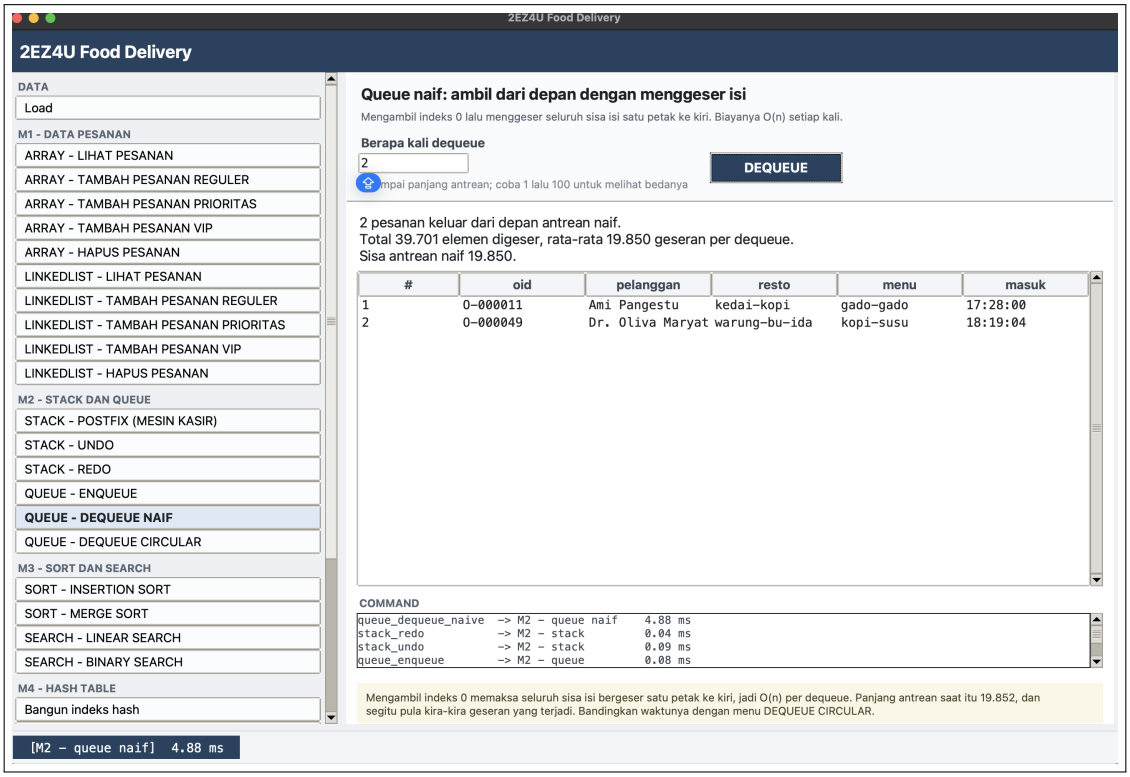
> stack_undo
  100 pesanan dikembalikan ke DEPAN antrean circular   [M2 - stack]  0,04 ms
> stack_redo
  100 pesanan keluar lagi dari antrean circular        [M2 - stack]  0,03 ms
  tumpukan undo 3, redo 0

> queue_enqueue   Pencabang           (aksi BARU sesudah undo)
> stack_redo
  tumpukan redo kosong, tidak ada yang diulang         [M2 - stack]  0,01 ms
```

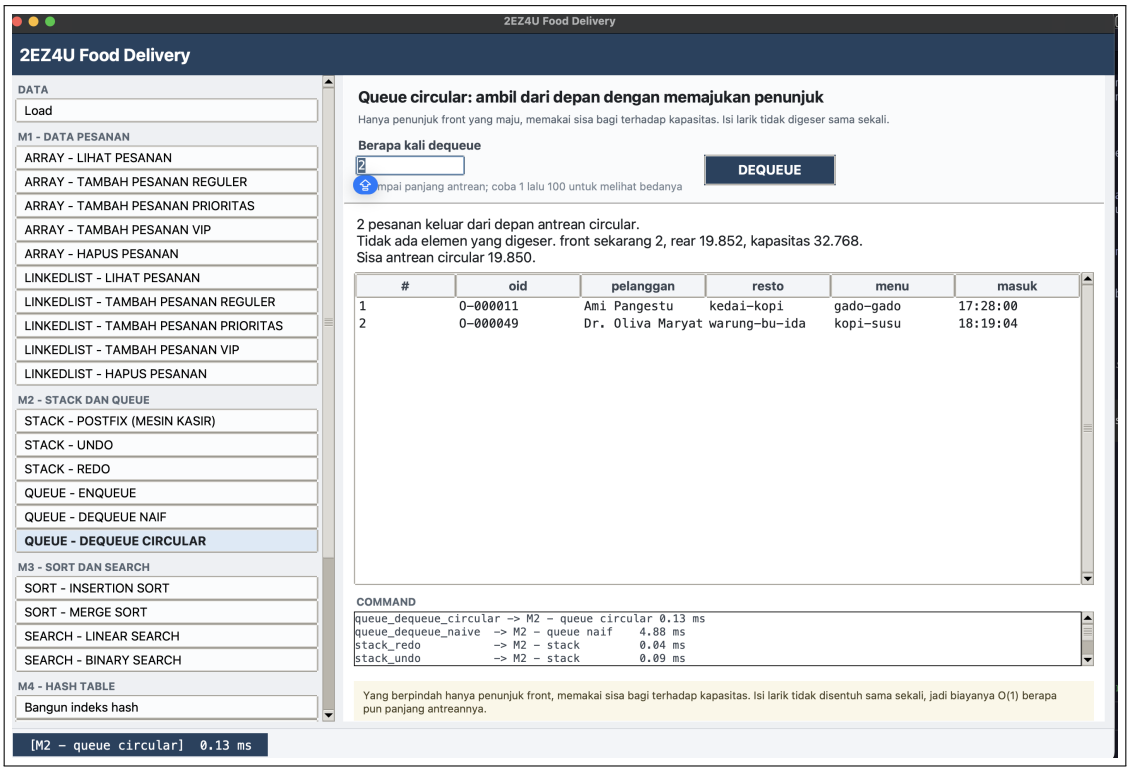
Angka di atas diambil dari implementasi acuan pada data 200.000 baris (19.850 pesanan berstatus `ANTRE`). Yang dinilai bukan angka persisnya, melainkan **polanya**: `enqueue` sama murahnya di kedua versi, dan seluruh selisih muncul di `dequeue`. Tiga tangkapan layar berikut adalah transkrip yang sama, dilihat dari layar aplikasinya.




Menu `QUEUE - ENQUEUE` menambahkan satu pesanan ke belakang **kedua** antrean sekaligus, sehingga panjangnya tetap sama, 19.852 dan 19.852. Perhatikan panel `COMMAND` di bawah: `queue_enqueue` hanya 0,08 ms dan 0,03 ms, sama murahnya pada versi naif maupun circular karena keduanya cuma menempel di ujung belakang.




Menu `QUEUE - DEQUEUE NAIF`, dua kali dequeue pada antrean berisi 19.852 pesanan. Perhatikan baris hasilnya: 39.701 elemen digeser, rata-rata 19.850 geseran untuk *satu* pesanan yang dilayani, dan baris status di pojok kiri bawah mencatat 4,88 ms.




Menu `QUEUE - DEQUEUE CIRCULAR` pada data yang sama persis: dua pesanan yang keluar juga identik, `O-000011` dan `O-000049`. Bedanya ada di kalimat kedua, tidak ada elemen yang digeser, hanya `front` yang maju ke 2 sementara `rear` 19.852 pada kapasitas 32.768 – dan di panel `COMMAND`, 0,13 ms melawan 4,88 ms versi naif.

## Langkah kerja yang disarankan

1. Tulis `Tumpukan` (stack) di atas `Larik` milikmu: `push`, `pop`, `peek`, `kosong`. Semua M2 berdiri di atasnya.
2. Tulis pemecah token kasir. Angka berdigit banyak harus jadi **satu** token: `12000`, bukan `1` lalu `2`.
3. Tulis `ke_postfix`, uji dengan empat ekspresi: tanpa kurung, berkurung, ada `-` beruntun, dan kurung bersarang.
4. Tulis `hitung`, uji khusus dengan `-` dan `/` untuk memastikan urutan `pop`-nya benar.
5. Tulis `AntreanNaif` di atas `Larik`: `enqueue` menempel di belakang, `dequeue` mengambil indeks 0. Hitung dan simpan jumlah geserannya, angka itu yang nanti ditampilkan.
6. Tulis `AntreanMelingkar(K)`: `data`, `front`, `rear`, `count`. Uji dengan kapasitas kecil ($K = 4$): masukkan 3, keluarkan 2, masukkan 3 lagi, pastikan penunjuk **berputar** dan datanya tetap benar. Ini uji terpenting di M2.
7. Tangani dua kasus batas secara eksplisit: `dequeue` saat kosong, `enqueue` saat penuh (tumbuh atau tolak, putuskan dan dokumentasikan).
8. Isi kedua antrean dari data yang sama, lalu sambungkan enam menunya. Setiap aksi antrean di-`push` ke tumpukan undo **dan** mengosongkan tumpukan redo.
9. Tambahkan ke `stats()`: `queue_len` kedua versi, `queue_capacity`, `total_geseran`, `undo_depth`, `redo_depth`.

## 6.3 M3: Sort & Search

kerja mgg 6–9 · deadline saat sesi mgg 9 · 15%



### Ceritanya

Layanan tutup pukul 9 malam. Pemilik warung minta **laporan harian**: seluruh pesanan hari itu, *terurut menurut waktu selesai*, supaya terlihat jam-jam sibuknya. Lalu datang pertanyaan susulan: "pesanan O-4217 tadi selesai jam berapa?" Jawabannya harus cepat, bukan dengan membaca seluruh laporan dari atas.

Dua keterampilan klasik masuk di sini: **mengurutkan** dan **mencari**. Dan seperti M1 dan M2, masing-masing dikerjakan dengan **dua cara** supaya selisihnya terbaca, bukan dihafal.

### Empat algoritma yang harus ada setelah M3

- `insertion_sort(A, kunci)` bekerja di tempat dan **stabil**. $O(n^2)$.
- `merge_sort(A, kunci)` juga stabil, dan sanggup mengurutkan seluruh data. $O(n \log n)$.
- `linear_search(A, target)` menelusuri satu per satu, tidak menuntut data terurut. $O(n)$.
- `binary_search(A, target)` membuang separuh sisa tiap langkah, menuntut data terurut. $O(\log n)$.

**Setiap menu wajib melaporkan jumlah perbandingan, bukan hanya waktu.** Waktu dipengaruhi mesin, beban lain, dan keberuntungan. Jumlah perbandingan tidak: ia angka yang sama di komputer siapa pun, dan angka itulah yang membuktikan $O(n^2)$ versus $O(n \log n)$. Sediakan satu pencacah sederhana yang dititipkan ke algoritmanya.

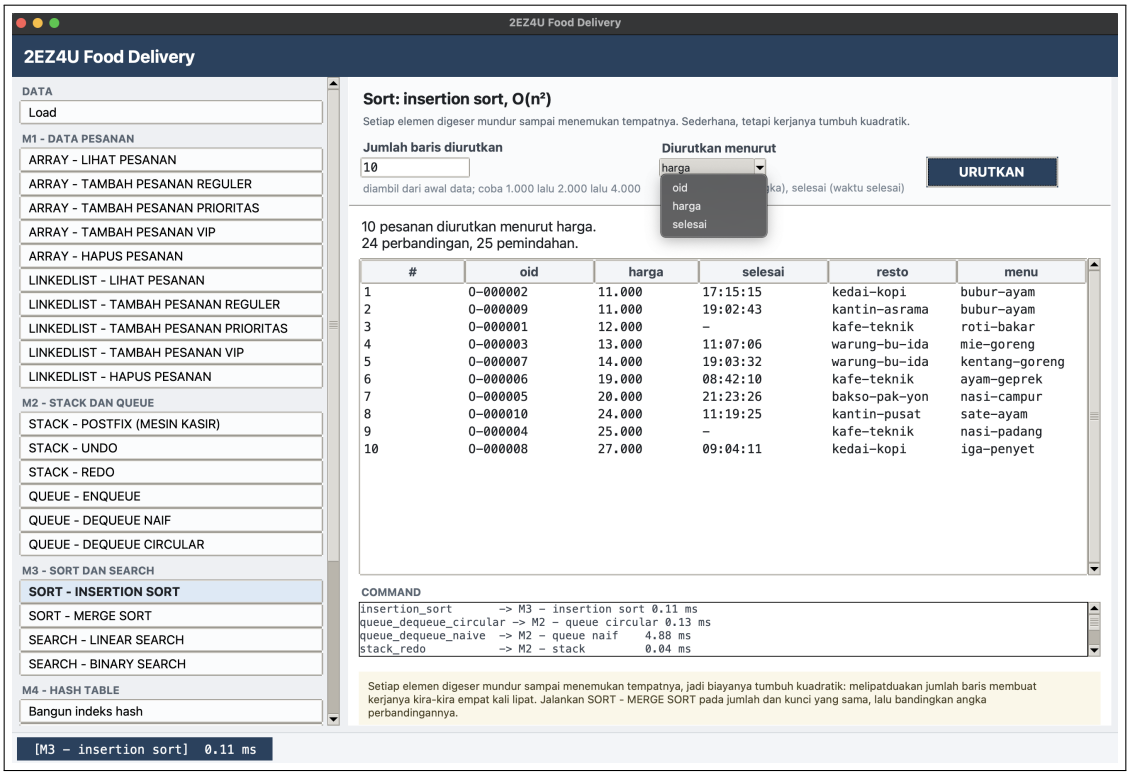
### Bagian A (1): insertion sort

Insertion sort meniru cara orang merapikan kartu di tangan: ambil kartu berikutnya, geser ke kiri melewati kartu-kartu yang lebih besar, letakkan di tempatnya.

| | | | | | | |
|---|---|---|---|---|---|---|
| **1.** | 08.05 | 08.12 | 08.30 | 08.19 | 08.41 | *ambil **08.19**; bandingkan ke kiri* |
| **2.** | 08.05 | 08.12 | – | 08.30 | 08.41 | *08.30 lebih besar ⇒ digeser ke kanan* |
| **3.** | 08.05 | 08.12 | 08.19 | 08.30 | 08.41 | *08.12 lebih kecil ⇒ berhenti, sisipkan* |

Kekuatannya: sangat sederhana, dan pada data yang **hampir terurut** ia berjalan mendekati $O(n)$ karena hampir tidak ada yang perlu digeser. Kelemahannya muncul begitu data besar dan acak: rata-rata separuh bagian kiri harus digeser untuk setiap elemen, sehingga kerjanya tumbuh kuadratik.

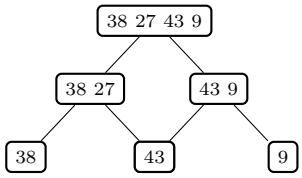
**Stabil itu apa?** Pengurutan disebut stabil bila dua data dengan kunci sama tetap mempertahankan urutan aslinya. Dua pesanan yang selesai pada menit yang persis sama harus tetap muncul sesuai urutan masuknya. Dalam kode, ini berarti syarat penggeseran memakai **lebih besar** ($>$), bukan **lebih besar atau sama dengan** ($\geq$).




Menu `SORT - INSERTION SORT` pada 10 baris pertama dengan kunci `harga`; daftar pilihan kuncinya (`oid`, `harga`, `selesai`) sedang terbuka. Perhatikan dua baris teratas: keduanya berharga 11.000 dan tetap muncul berurutan `O-000002` lalu `O-000009` seperti urutan asalnya, itulah bukti stabil. Untuk 10 baris saja sudah perlu 24 perbandingan dan 25 pemindahan.

### Bagian A (2): merge sort

Merge sort membelah data menjadi dua sampai tersisa satu elemen, lalu *menggabungkan* kembali dua bagian yang sudah terurut. Karena kedalaman pembelahannya $\log n$ dan tiap tingkat memerlukan $O(n)$ penggabungan, biayanya $O(n \log n)$, dan itu **tidak bergantung** pada data awalnya seperti apa.

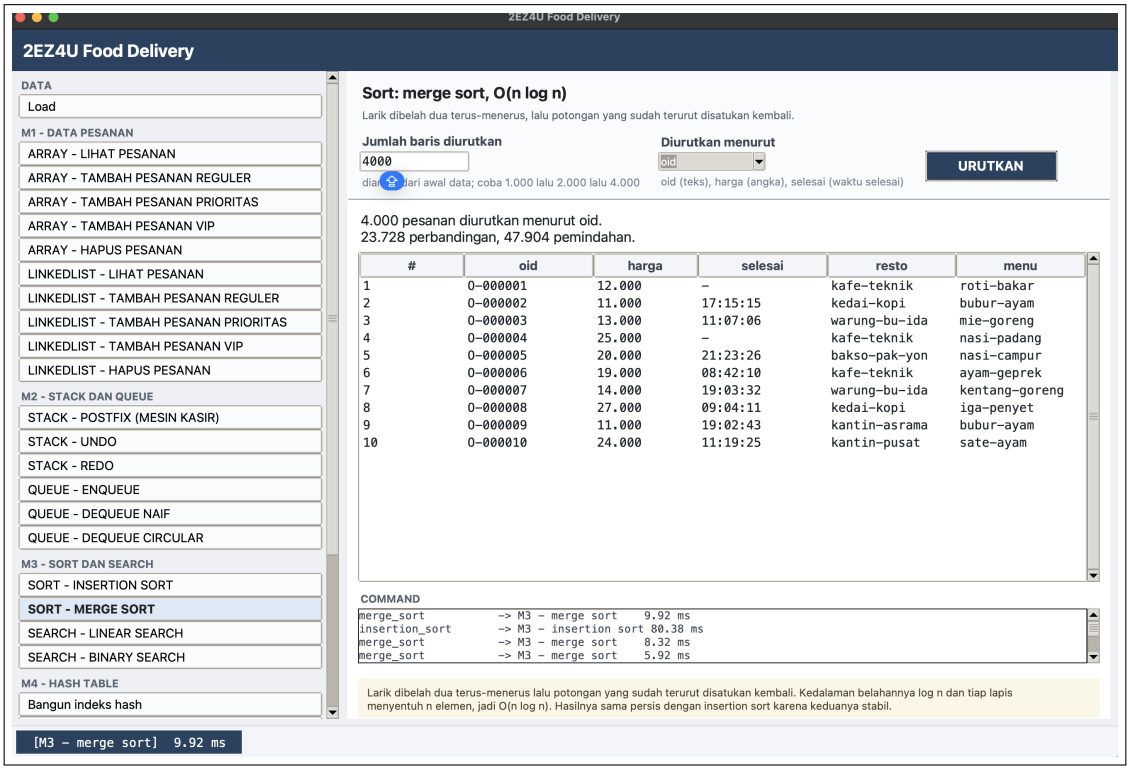



**belah** sampai tersisa satu elemen

lalu **gabung** berpasangan:
27 38 $\cup$ 9 43 $\rightarrow$ 9 27 38 43

Bagian penggabungan menyimpan dua kesalahan langganan. **Pertama**, sisa salah satu bagian tidak ikut disalin setelah perulangan utama berhenti, sehingga beberapa data hilang di ujung. **Kedua**, syarat perbandingannya memakai $<$ sehingga pengurutannya tidak lagi stabil; pakai $\leq$ agar elemen dari bagian kiri selalu didahulukan saat kuncinya sama.

| Algoritma | Rata-rata | Terbaik | Dipakai untuk |
|---|---|---|---|
| Insertion sort | $O(n^2)$ | $O(n)$ (hampir terurut) | data kecil atau hampir terurut |
| Merge sort | $O(n \log n)$ | $O(n \log n)$ | data besar, $10^5$ baris ke atas |




Menu `SORT - MERGE SORT` pada 4.000 baris dengan kunci `oid`: 23.728 perbandingan dan 47.904 pemindahan. Perhatikan panel `COMMAND` di bawah, di situ kedua sort berjejer pada layar yang sama: `insertion_sort` 80,38 ms sedangkan `merge_sort` 9,92 ms.

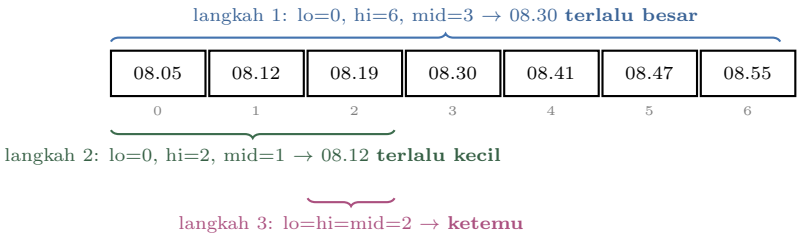
### Bagian B (1): linear search

Telusuri dari depan, bandingkan satu per satu, berhenti begitu ketemu. Tidak ada syarat apa pun: data boleh berantakan. Kasus terburuk, yaitu data tidak ada, menyentuh seluruh $n$ elemen.

Sederhana bukan berarti salah. Untuk pencarian yang hanya dilakukan sekali dua kali, linear search justru pilihan yang benar, karena alternatifnya menuntut pengurutan yang jauh lebih mahal daripada satu kali penelusuran.

### Bagian B (2): binary search

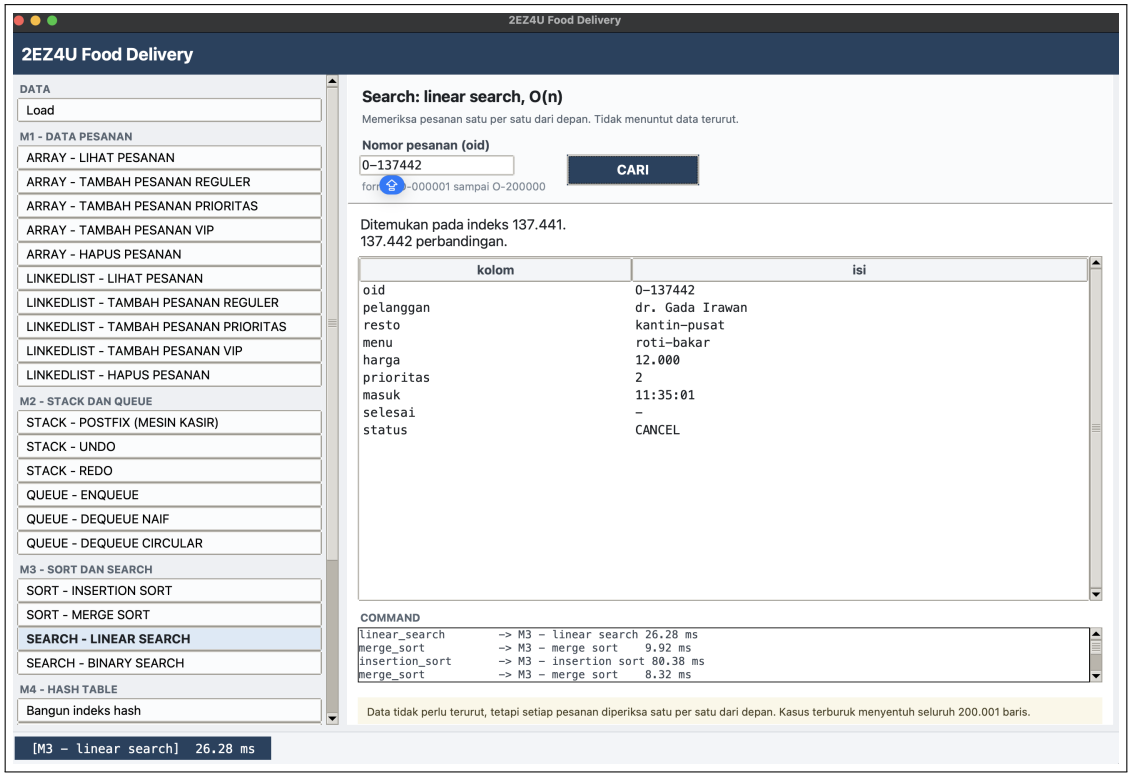
Pada data terurut, setiap perbandingan membuang *separuh* sisa kemungkinan. Sepuluh perbandingan sudah cukup untuk seribu data; dua puluh untuk sejuta.



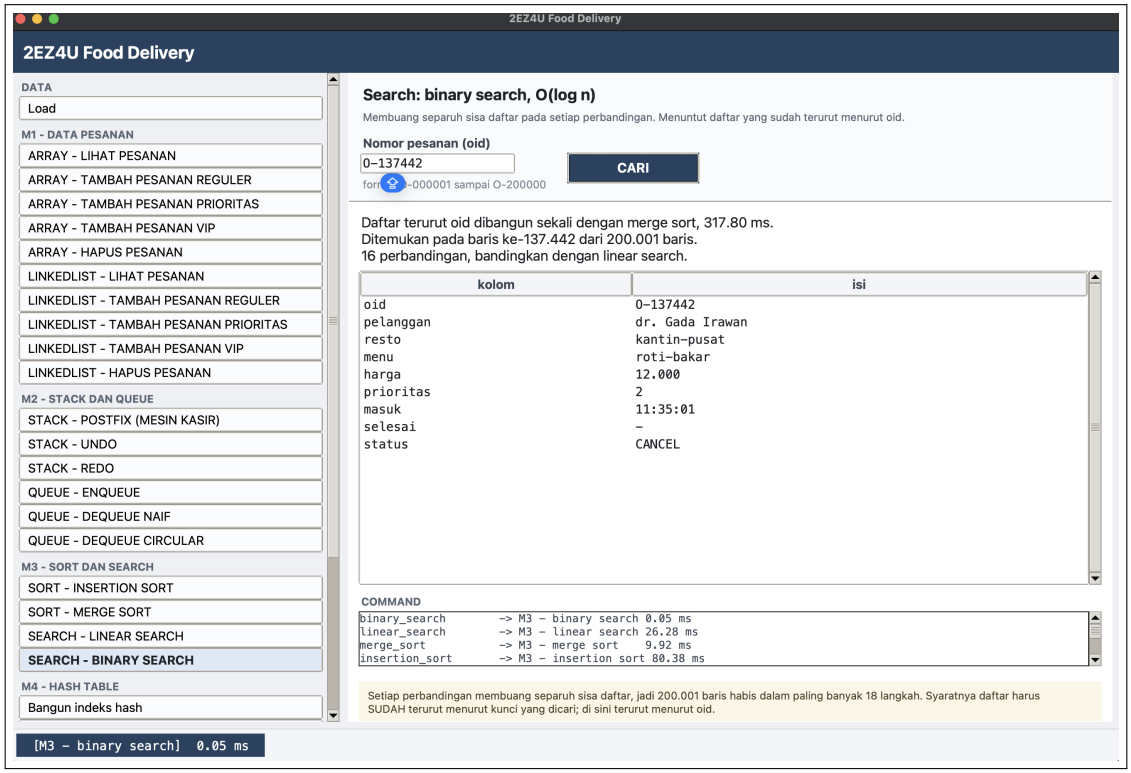
```
FUNGSI cari(A, target):
    lo <- 0 ; hi <- panjang(A) - 1
    SELAMA lo <= hi:                        # perhatikan: <= , bukan <
        mid <- (lo + hi) // 2
        JIKA A[mid] == target : KEMBALIKAN mid
        JIKA A[mid] <  target : lo <- mid + 1
        SELAIN ITU            : hi <- mid - 1
    KEMBALIKAN -1                           # tidak ditemukan
```

**Binary search tidak gratis.** Ia menuntut daftar yang sudah terurut *menurut kunci yang dicari*. Laporan harian diurutkan menurut waktu selesai, jadi mencari `oid` di sana **tidak sah**: siapkan daftar terpisah yang terurut menurut `oid`. Dan pengurutan itu sendiri lebih mahal daripada satu kali linear search, sehingga binary search baru untung bila daftarnya dicari berkali-kali.

Dua tangkapan layar berikut mencari `oid` yang sama persis pada data yang sama persis, satu dengan tiap cara.




Menu `SEARCH - LINEAR SEARCH` mencari `O-137442` di antara 200.001 baris. Perhatikan baris hasil: 137.442 perbandingan, satu untuk setiap baris yang dilewati sampai targetnya ketemu di indeks 137.441, dan waktunya 26,28 ms.




Menu `SEARCH - BINARY SEARCH` mencari `O-137442` yang sama dan menemukan baris yang sama, tetapi cukup dengan **16** perbandingan dan 0,05 ms. Perhatikan baris pertama hasilnya, di situ ongkos yang tidak gratis itu tercatat terang-terangan: daftar terurut `oid` harus dibangun sekali lebih dulu dengan merge sort, 317,80 ms.

### Contoh nyata

```
> insertion_sort   jumlah=2000   kunci=harga
  2.000 pesanan terurut | 962.430 perbandingan  [M3 - insertion sort] 163,26 ms
> merge_sort       jumlah=2000   kunci=harga
  2.000 pesanan terurut |  19.240 perbandingan  [M3 - merge sort]       6,36 ms

> insertion_sort   jumlah=4000   kunci=harga     (n dilipatduakan)
  4.000 pesanan terurut | 3.831.520 perbandingan [M3 - insertion sort] 660,36 ms
> merge_sort       jumlah=4000   kunci=harga
  4.000 pesanan terurut |    42.355 perbandingan [M3 - merge sort]      13,66 ms

> merge_sort       jumlah=200000 kunci=harga
  200.000 pesanan terurut | 3.226.139 perbandingan [M3 - merge sort]   1,13 s
> insertion_sort   jumlah=200000
  DITOLAK: batasnya 20.000 baris, pakai merge sort untuk data penuh

> linear_search    O-199999
  ditemukan di indeks 199.998 | 199.999 perbandingan [M3 - linear]  24,52 ms
> binary_search    O-199999
  daftar terurut oid dibangun sekali (merge sort)              562,58 ms
  ditemukan di baris 199.999 | 17 perbandingan       [M3 - binary]   0,03 ms
```

Yang dinilai bukan angka persisnya, melainkan **polanya**: begitu $n$ dilipatduakan, perbandingan insertion sort menjadi kira-kira **empat kali**, sedangkan merge sort hanya sekitar **dua kali**. Dan linear search memakai belasan ribu kali lebih banyak perbandingan daripada binary search pada data yang sama.

**Perhatian: tag `m3` adalah tiket masuk sesi Demo Progress minggu 9.** Tanpa tag tersebut, soal sesi tidak dapat dibuka. Submit lebih awal. Jangan menunggu jam terakhir.

### Checklist siap submit M3

- ☐ Keempat algoritma ada, dan masing-masing melaporkan jumlah perbandingannya.
- ☐ Kedua sort terbukti stabil, dan hasilnya identik baris per baris pada data yang sama.

☐ Merge sort benar pada data acak, terurut, terbalik, dan berisi duplikat.

☐ Binary search benar pada empat kasus (tengah, kiri, kanan, tidak ada) + larik kosong.

☐ Menu insertion sort menolak data terlalu besar dengan pesan, bukan membeku.

☐ Ada tabel $n$ versus perbandingan di README untuk kedua sort, lengkap dengan grafiknya.

☐ Alasan memilih tiap algoritma tertulis dan bisa kamu jelaskan lisan.

☐ Tes M1–M2 masih hijau; tag `m3` sudah di-push **sebelum** sesi minggu 9.